

OPERATORS

Definition:

Operators are pre-defined symbols in Java that are used to perform specific tasks. The most basic operators are from arithmetic operations performed in school days. Let us consider an expression for example.

$$15 + 25 = 40$$

In the above example, + is the addition operator that returns the sum of 15 and 25 to give 40. Here, 15 and 25 are the data on which addition operation is performed. There are some basic terminologies.

Operation: The task or action that is performed (addition)

Operator: The symbol used for performing the task (+)

Operand: The data or value on which operation is performed (15 and 25)

Characteristics of an Operator:

There are three important characteristics for an Operator. They are:

1. Return the data after operation
2. Precedence
3. Associativity

Return the data after operation:

Every operator will perform the pre-defined operation and return the result. The result can be stored in a variable container for later usage. To create an appropriate container, we should know

- ❖ What type of data is returned
- ❖ What value/data is returned

Precedence:

If an expression contains more than one operator, then there should be a priority in the operator execution. That **Priority** is called Precedence.

Eg: $12 * 2 + 10$

In the above expression, if * is executed first, result is 34. But if + is executed first, result is 144. How will the expression get executed? Thus, there is a rule called BODMAS which is followed. As per that rule, * has higher priority compared to +. The result will be 34.

Associativity:

As per the BODMAS rule + and – have same priority, and * and / have same priority. What will happen in this scenario if an expression has these operators together. Associativity defines the **order of execution** for the operators. The rule for + and – is execution from left to right.

Eg: $12+5-6-3+4$

The above expression will be executed from left to right and the result will be 12.

TYPES OF OPERATORS:

Operators are of two classifications. They are:

Based on Operands:

- ❖ Unary Operator - uses only one operand
- ❖ Binary Operator - uses two operands
- ❖ Ternary Operator - uses three operands

Based on Operation/Task performed:

- ❖ Arithmetic Operators
- ❖ Relational Operators
- ❖ Logical Operators
- ❖ Assignment Operator
- ❖ Compound Assignment Operators
- ❖ Bitwise Operators
- ❖ Miscellaneous Operators, like typecast and instance of
- ❖ Conditional Operator
- ❖ Increment and Decrement Operators

ARITHMETIC OPERATORS:

Arithmetic operators are binary operators. There are five Arithmetic operators which are used to perform basic mathematical operations. They are:

- Plus operator (+) : addition and concatenation
- Minus operator (-) : subtraction
- Asterisk operator (*) : multiplication
- Slash operator (/) : division, returns quotient
- Modulus operator (%) : division, returns remainder

Plus Operator (+):

This operator is polymorphic in nature, which means it can exhibit two different functions based on the operands.

When both operands are integers, it returns the sum of both values. If one or both operands are String, it is used to concatenate or merge both data.

Eg: $10 + 20$ gives 30

But, $10 + \text{"hello"}$ gives 10hello, due to concatenation

Similarly, "good" + "morning" gives goodmorning

ASSIGNMENT AND COMPOUND ASSIGNMENT OPERATORS:

Assignment operator is used to assign the value on the RHS to variable present on the LHS.

Eg: `int a = 12;`

The value 12 is assigned to variable a.

Compound Assignment operators are used to combine Arithmetic and Assignment operators. This operator is used to perform the operation and assign the value to the same variable. They are of five types as follows:

- Plus operator (+=) : addition
- Minus operator (-=) : subtraction
- Asterisk operator (*=) : multiplication
- Slash operator (/=) : division, returns quotient
- Modulus operator (%=) : division, returns remainder

Eg: `int a = 6;`

`a += 5;`

The above expression works as (`a = a+5;`), which means 6+5 is performed and the result 11 is stored in the variable a itself. Other Compound Assignment operators work in the same manner.

RELATIONAL OPERATORS:

Relational operators are used to perform comparison operations. The return type of Relational operators is always Boolean, which means true or false. They are binary operators. They are of six types as follows:

- ❖ Equality Operator (==)
- ❖ Inequality Operator (!=)
- ❖ Greater Than Operator (>)
- ❖ Lesser Than Operator (<)
- ❖ Greater Than or Equal To Operator (>=)
- ❖ Lesser Than or Equal To Operator (<=)

Equality Operator:

It is used to compare two values and return true if they are equal or false if they are not equal.

Eg: $10==10$ gives true

$12==45$ gives false

Inequality Operator:

It is used to compare two values and return true if they are not equal or false if they are equal.

Eg: $12!=15$ gives true

$15!=15$ gives false

Greater Than Operator:

It is used to find if left operand is greater than right operand.

Eg: $10>5$ gives true

$3>48$ gives false

Lesser Than Operator:

It is used to find if left operand is smaller than right operand.

Eg: $6<48$ gives true

$78<15$ gives false

Greater Than or Equal To Operator:

It is used to find if left operand is greater or equal to right operand.

Eg: $10>=6$ gives true

$10>=10$ gives true

$10>=48$ gives false

Lesser Than or Equal To Operator:

It is used to find if left operand is smaller or equal to right operand.

Eg: $8<=15$ gives true

$8<=8$ gives true

$8<=4$ gives false

LOGICAL OPERATORS:

Logical Operators are used when multiple conditions are to be merged to get the final result. They take input as Boolean values and return the result in Boolean values, that is either true or false. There are three Logical operators as follows:

- Logical Unary Operator, called as Logical NOT (!)
- Logical Binary Operators, like
 - ❖ Logical AND (&&)
 - ❖ Logical OR (||)

Logical AND (&&):

It evaluates conditions that return Boolean data.

If the first condition is false, it does not check second condition. The final result is false.

If the first condition is true, it checks for the second condition. If the second condition is false, final result is false. If the second condition is true, final result is true.

The AND operator can be expressed in the following truth table:

FIRST CONDITION	SECOND CONDITION	FINAL RESULT
FALSE	NOT CHECKED	FALSE
TRUE	FALSE	FALSE
TRUE	TRUE	TRUE

Logical OR (||):

It evaluates conditions that return Boolean data.

If the first condition is true, it does not check second condition. The final result is true.

If the first condition is false, it checks for the second condition. If the second condition is true, final result is true. If the second condition is false, final result is false.

The OR operator can be expressed in the following truth table:

FIRST CONDITION	SECOND CONDITION	FINAL RESULT
TRUE	NOT CHECKED	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Logical NOT (!):

It negates or gives opposite of what Boolean data is passed. If input is true, it returns false. If input is false, it returns true.

The NOT operator can be expressed in the following truth table:

INPUT	OUTPUT
TRUE	FALSE
FALSE	TRUE

CONDITIONAL OPERATOR:

Conditional Operator in Java is the example for ternary operator, which can take three operands. The first operand is in the form of condition, that returns a Boolean value. The second and third operands are statements/values/expressions. The syntax for conditional operator is:

Syntax:

CONDITION ? STATEMENT1 : STATEMENT2

If the output of the condition is true, statement 1 is executed and if the output of the condition is false, statement 2 is executed. Statements 1 and 2 should be of same datatype.

The final result of the Conditional operator can be stored in a variable for future usage or printing on the console. The programmer should know what will be the datatype of the final result in order to create the variable container in that type.

Eg: int a = 12;
 int b = 15;
 a>b ? "yes" : "no"

The above conditional operator returns data of String type. Hence, the variable to store the final result should be of String type.

String result = a>b ? "yes" : "no";

The above statement on execution will evaluate the condition. Since the condition returns false, the statement 2 will be executed and "no" will be stored in variable result.

CONDITIONAL OPERATOR EXAMPLE PROGRAM:

```
1  class ClassName
2  {
3      public static void main(String[] args)
4      {
5          int a = 10;
6          int b = 20;
7          String res = a>b ? "a is greater" : "b is greater";
8          System.out.println(res);
9      }
10 }
```

INCREMENT & DECREMENT OPERATORS:

These are Unary operators that are used to increase or decrease the value of variable by one unit. There are two types in each operator based on when they perform the operation as pre and post.

Post-Increment and Post-Decrement Operators:

The existing value of the variable is used.

The value is increased or decreased by one.

Pre-Increment and Pre-Decrement Operators:

The value is increased or decreased by one.

The new value is used.

Post-Increment Operator:

```
int a = 4;
```

```
System.out.println(a++);
```

The value of a, that is 4 is printed. The value is increased to 5 in the memory block, for future usage.

Pre-Increment Operator:

```
int a = 4;
```

```
System.out.println(++a);
```

The value is increased to 5 and printed. The same value 5 is available for future usage.

Post-Decrement Operator:

```
int a = 4;
```

```
System.out.println(a--);
```

The value of a, that is 4 is printed. The value is decreased to 3 in the memory block, for future usage.

Pre-Decrement Operator:

```
int a = 4;
```

```
System.out.println(--a);
```

The value is decreased to 3 and printed. The same value 3 is available for future usage.

INCREMENT DECREMENT OPERATOR EXAMPLE PROGRAM:

```
1 package javapackage;
2
3 class Sample1 {
4
5     public static void main(String[] args) {
6         int a = 10;
7         int b = a++ + ++a;
8         int c = a-- + --b + ++a + a++;
9         System.out.println(a);
10        System.out.println(b);
11        System.out.println(c);
12    }
13 }
14
15
```

<terminated> Sample1 [Java Appli

13
21
57